Decisioni

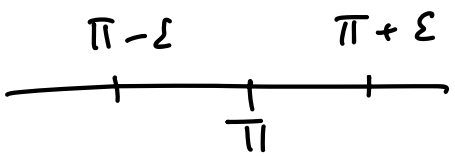
```
if ( condizione ) {
    // fai qualcosa
}
```

```
if (x > 0) ?
if (x == 0) ?
if (x != 10) }
```

```
double x;
scanf("%lf", &x);
double a = M_PI, epsilon = 0.0001;
```

```
if ( fabs(x - a) <= epsilon ) {
```

~~if (x == M_PI)~~



Decisioni: Composte

probabilità: $p \in [0,1]$

$0 <= p <= 1$

probabilita.c

```c
#include <stdio.h>

int main() {
    double p;
    printf("p = ");
    scanf("%lf", &p);
    if (0 <= p <= 1) {
        printf("p valida\n");
    } else {
        printf("p non valida\n");
    }
}
```

printf("p in [0,1]: ");

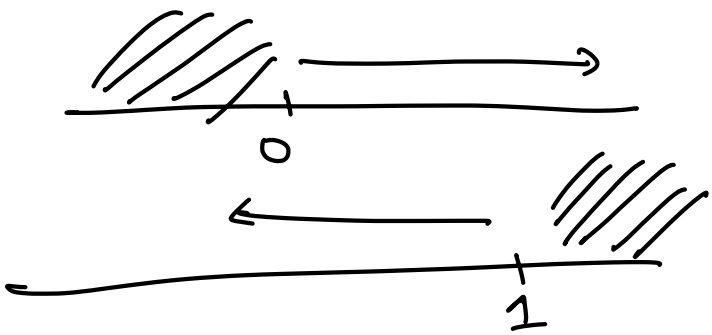
0 <= p : 1 Vero
0 Falso

0,1 <= 1 sempre

Terminale: probabilita.c

```
$ gcc -Wall -Wextra -o app.exe probabilita.c -lm
probabilita.c:7:16: warning: result of comparison of constant 1 with boolean
    expression is always true [-Wtautological-constant-compare]
$ ./app.exe
p = 0.3
p valida
$ ./app.exe
p = 5
p valida
$ ./app.exe
p = -2
p valida
```

$0 <= p <= 1 \equiv 0 <= p$ AND $p <= 1$



AND &&

$0 <= p$ && $p <= 1$

$p >= 0$ && $1 >= p$

probabilita.c

```c
#include <stdio.h>

int main() {
    double p;
    printf("p = ");
    scanf("%lf", &p);

    if (p >= 0 && p <= 1) {
        printf("p valida\n");
    } else {
        printf("p non valida\n");
    }

    int dentro = (p >= 0 && p <= 1);
    int fuori = (p < 0 || p > 1);
    printf("dentro = %d, fuori = %d, !dentro = %d\n", dentro, fuori, !dentro);
}
```

if (cond1 && cond2) { }

if (dentro) { }

&& AND

|| OR logico

A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

negazione: !

```
if (dentro) {
    ≡
}
```

```
if (!fuori) {
    ≡
}
```

```
if ( x<0 && x>1 )
```

AND errato

!=

operatore negazione logica

!0 = 1

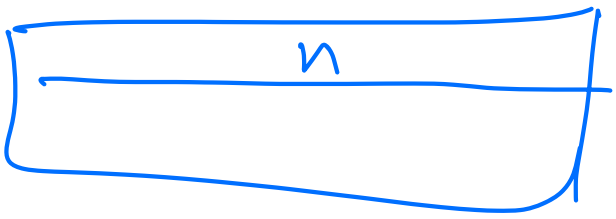
```
int dentro = 1;
int fuori = !dentro;
printf("dentro: %d \n", dentro);
printf("fuori: %d \n", fuori);
```

Regola di De Morgan:

!(A AND B) = !A OR !B

!(A && B) !A || !B

Numeri pari




```
int n = 5;
int ris = n / 2;

int resto = n % 2;
```

% operatore resto

```
int pari = resto == 0;
```

resto == 0 ; — confronto

= — assegnazione

```
int dispari = resto != 0 ✓
```

~~resto == 1~~

```
n = -5;
if ( n % 2 == 0 ) { // pari
} else {
    // dispari
}
```

resto

multiplo di 10 %

```
if ( n % 10 == 0 )
```

Operatore resto %

int r = n % m

$r \in [0, 1, 2, 3, \dots, m-1]$

numeri 0, ..., 7 => n % 8

Multiplo di 3 e 5

n % 15

(n % 3 == 0 && n % 5 == 0)

Anno Bisestile

- divisibile per 4
- ~~non~~ divisibile 400
- non divisibile per 100

divisibilita.c

```c
#include <stdio.h>

int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);

    if (n % 2 == 0) {
        printf("%d pari\n", n);
    } else {
        printf("%d dispari\n", n);
    }

    if (n % 3 == 0 && n % 5 == 0) {
        printf("multiplo di 3 e 5\n");
    } else if (n % 3 == 0 || n % 5 == 0) {
        printf("multiplo di 3 o 5\n");
    } else {
        printf("nessun multiplo\n");
    }

    if ((n % 4 == 0 && n % 100 != 0) || n % 400 == 0) {
        printf("anno %d bisestile\n", n);
    } else {
        printf("anno %d non bisestile\n", n);
    }
}
```

```
Terminale: divisibilita.c

$ gcc -Wall -Wextra -o app.exe divisibilita.c -lm
$ ./app.exe
n = 2024
2024 pari
nessun multiplo
anno 2024 bisestile
$ ./app.exe
n = 1900
1900 pari
multiplo di 3 o 5
anno 1900 non bisestile
$ ./app.exe
n = 2000
2000 pari
multiplo di 3 o 5
anno 2000 bisestile
```

Ordine di Operatori

priorita.c

```c
#include <stdio.h>

int main() {
    double a = 4, b = 3, c = 2;
    printf("a*b/c   = %f\n", a * b / c);
    printf("a/b*c   = %f\n", a / b * c);
    printf("a/(b*c) = %f\n", a / (b * c));
}
```

6

0.6666 - -

```
Terminale: priorita.c

$ gcc -Wall -Wextra -o app.exe priorita.c -lm
$ ./app.exe
a*b/c   = 6.000000
a/b*c   = 2.666667
a/(b*c) = 0.666667
```

* e / pari ordine
si legge da sinistra a destra
=> usare le parentesi

$(a)/(b)$ $\quad a/b$ } parentesi inutili

$(a)*(b)/(c)$ } la stessa cosa

$(a*b)/c$

$a*(b/c)$ } non la stessa cosa

$(1.)/b$ inutile

$1./(b*c)$ } non la stessa cosa

$1./b*c$

$!A \quad -a$ come -3

ordine operatori:

$!A$

int A = -3;

$-A \rightarrow 3$

int A = 0;

$!A = 1$

$*$ $/$ $\%$

$+$ $-$

$<$ $<=$ $>$ $>=$

$==$ $!=$

$||$

$=$

$a - b$; operatore binario sottrazione.

$-a-b$

↳ operatore binario

↳ operatore unario

$-3-5$